Recurrent Machine Learning Models in Grocery Price Prediction

- Group Members
  - Arjun Yadapadithaya
  - Robert Sirkle
  - Sam Bean

Problem Statement

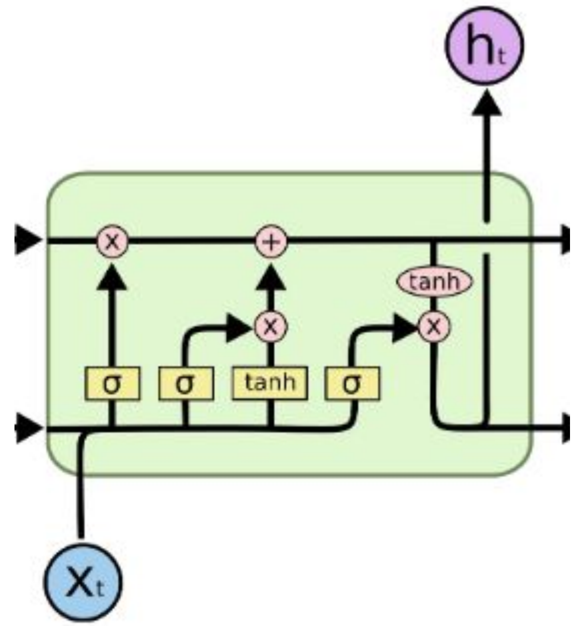
- Grocery demand forecasting is a highly complex, non-linear time-series problem.
- Sales are heavily influenced by external factors (holidays, promotions, oil prices).
- Traditional statistical models struggle to capture these long-term dependencies.
- Objective: Accurately predict daily unit sales across multiple stores using the Corporación Favorita dataset.

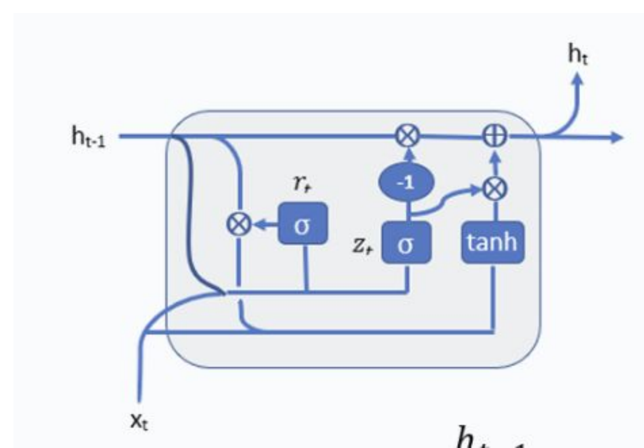
Motivation

- Accurate forecasting is critical for efficient inventory management.
- Overstocking: Leads to perishable food waste and high storage costs.
- Understocking: Causes lost revenue and customer dissatisfaction.
- Goal: Drive both financial efficiency and environmental sustainability using advanced sequence modeling.

Background

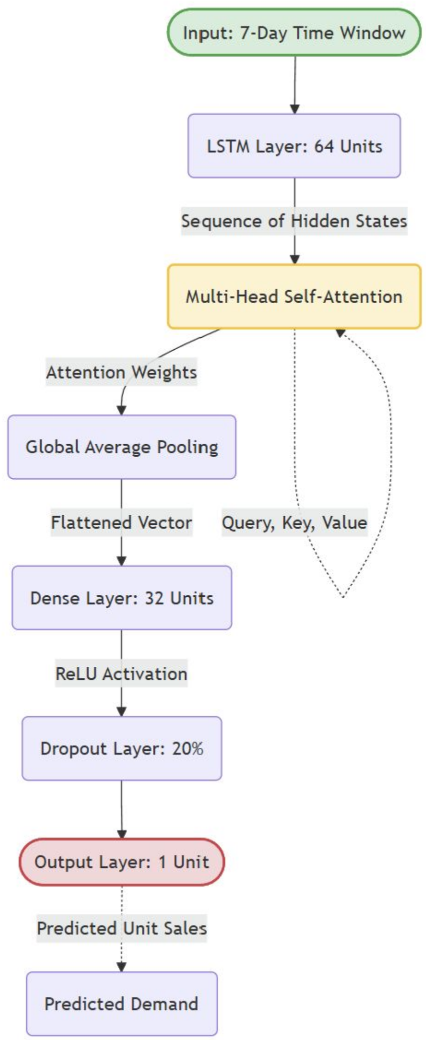
- Related Works
  - Hybrid LSTM-CNN with Attention Mechanism for Accurate and Scalable Grocery Sales Forecasting" M. S. Alam Bhuiyan, et al.
  - Kaggle's Corporación Favorita Grocery Sales Forecasting Competition
- Key concepts
  - Recurrent Networks
    - Keep memory of previous inputs through hidden states
    - GRU





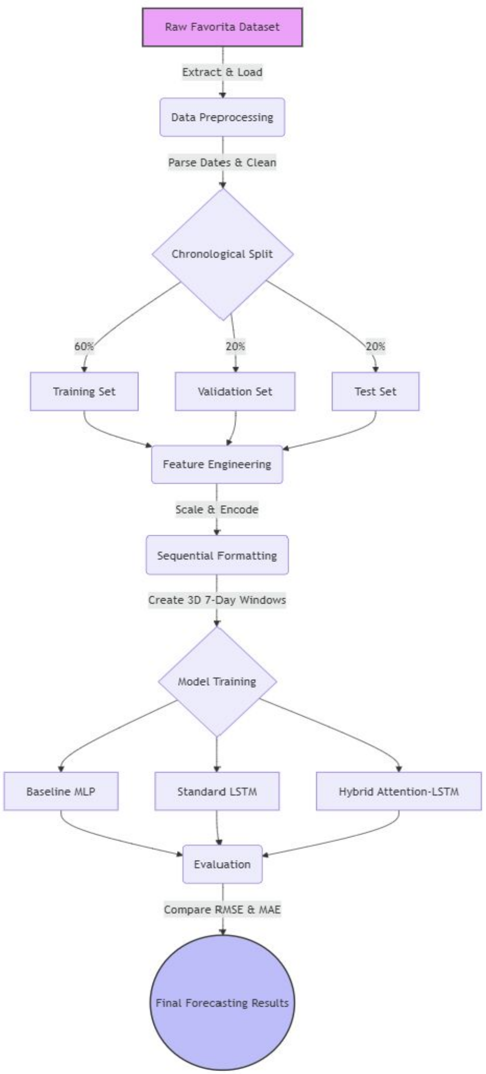
Method Overview

- Framed as a multivariate time-series regression task.
- Evaluated and compared three distinct architectures:
- Baseline MLP: Establishes a performance floor (lacks temporal awareness).
- Gated Recurrent Unit (GRU): Captures sequential dynamics and mitigates the vanishing gradient problem.
- Hybrid Attention-LSTM (Proposed): Uses Multi-Head Attention to dynamically focus on highly relevant historical events (e.g., past promotions).



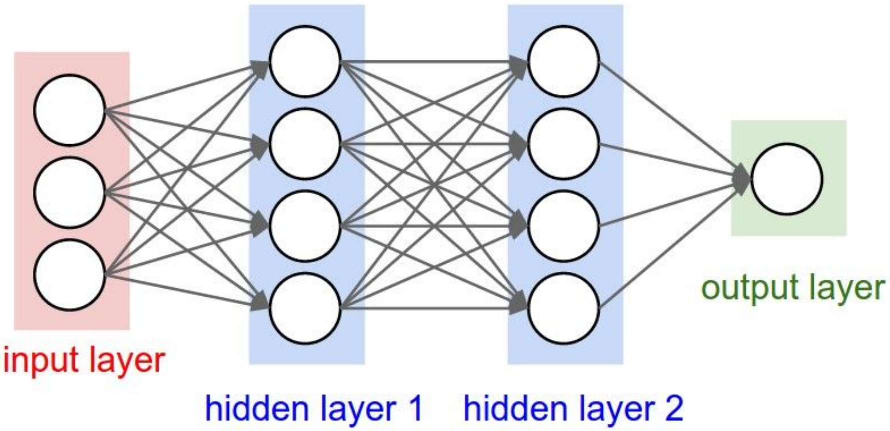
Pipeline / Workflow

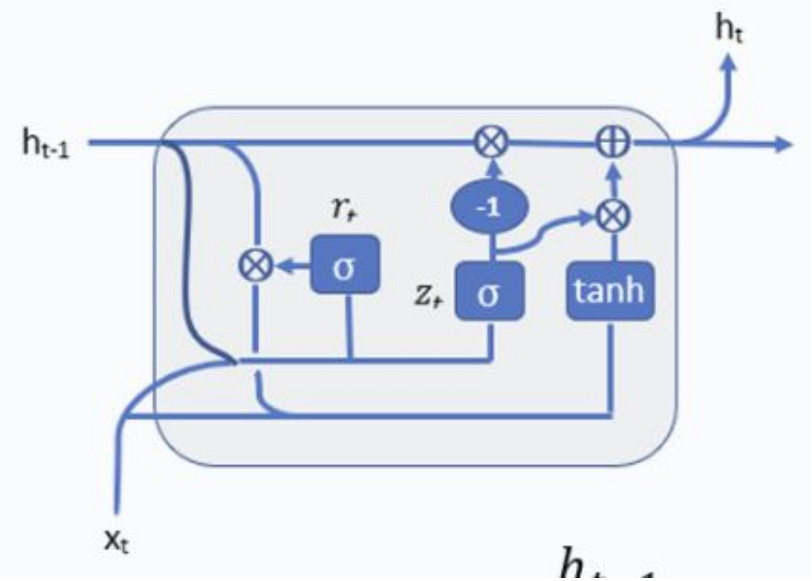
- Data Extraction: Automate Kaggle dataset processing.
- Preprocessing: Extract temporal features & handle missing values.
- Data Splitting: Chronological Train/Val/Test split to prevent leakage.
- Feature Engineering: Scale numericals & one-hot encode categories.
- Sequencing: Convert 2D tabular data into 3D 7-day time-windows.
- Training & Eval: Optimize models using Adam and compare using RMSE/MAE.



Model Details

- Models used
  - MLP
  - GRU
- Key parameters
  - MLP Parameters
  - GRU Parameters
    - Update Gate
    - Reset Gate
- Improvements
  - Recurrency from GRU





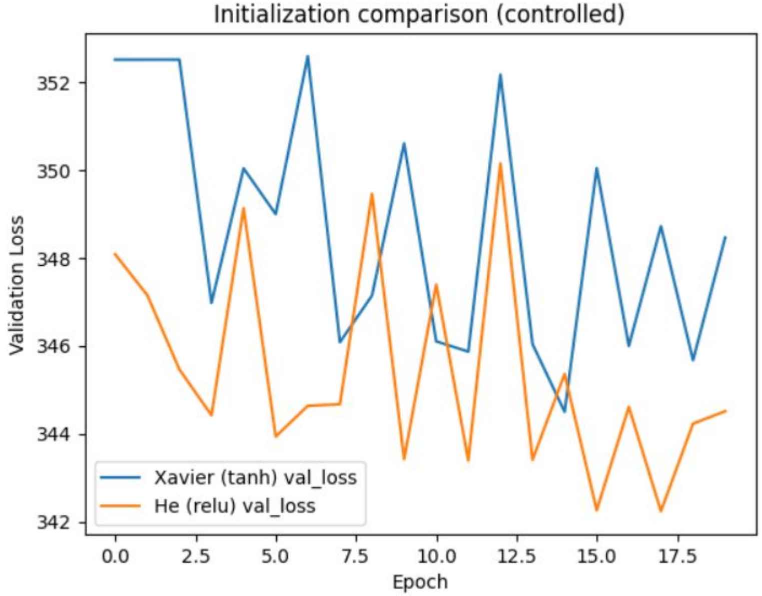
Experimental Setup

- Dataset
  - Taken from Kaggle’s “Corporación Favorita Grocery Sales Forecasting” competition
  - Data Loss
- Train/Test split
  - Train - 60%
  - Test - 20%
  - Validation - 20%
- Tracked Metrics
  - Mean Absolute Error
  - Mean Square Error

id	date	store_nbr	item_nbr	onpromotion
125497040	2017-08-16	1	96995	False
125497041	2017-08-16	1	99197	False
125497042	2017-08-16	1	103501	False
125497043	2017-08-16	1	103520	False
125497044	2017-08-16	1	103665	False
125497045	2017-08-16	1	105574	False
125497046	2017-08-16	1	105575	False
125497047	2017-08-16	1	105576	False
125497048	2017-08-16	1	105577	False
125497049	2017-08-16	1	105693	False
125497050	2017-08-16	1	105737	False
125497051	2017-08-16	1	105857	False
125497052	2017-08-16	1	106716	False
125497053	2017-08-16	1	108079	False
125497054	2017-08-16	1	108634	False

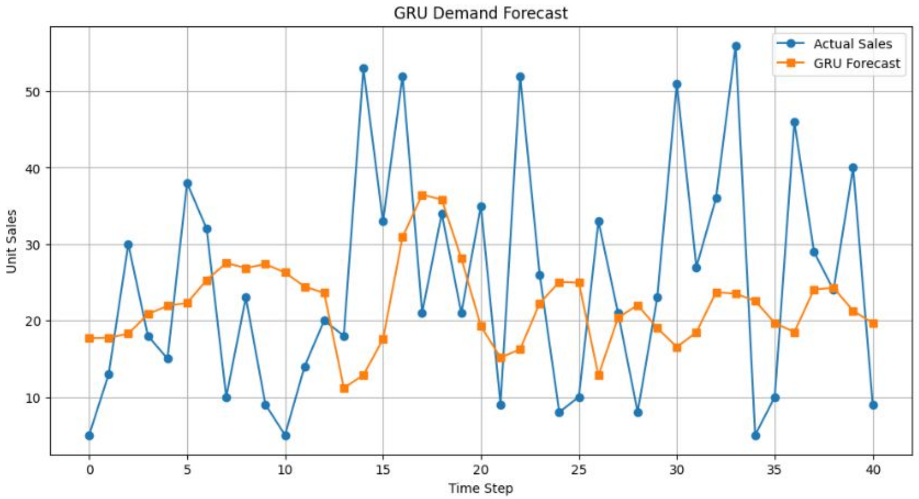
Results (Part 1)

MLP




```
Test Loss : 392.0069274902344
Test MAE  : 7.367404937744141
Test RMSE : 19.799165725708008
```

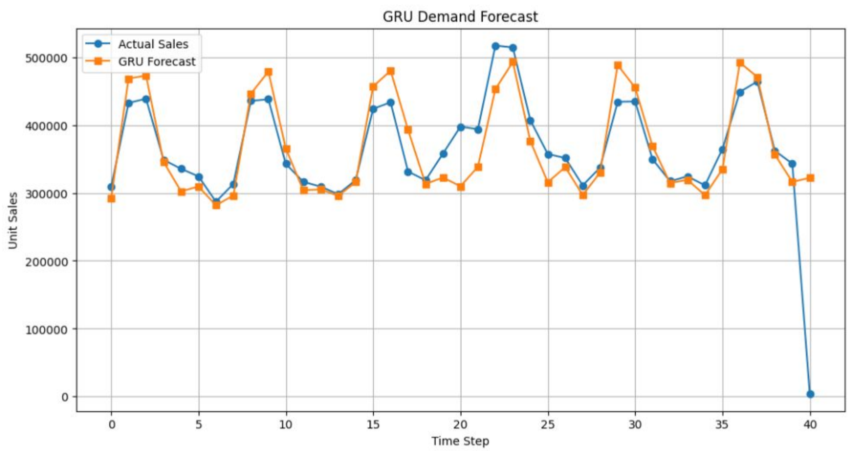
GRU: Single Store-Single Item with Holiday Considerations




```
GRU Test Loss : 0.003888986073434353
GRU Test MAE  : 0.0505838505923748
GRU Test RMSE : 0.062361735850572586
```

Results (Part 2)

GRU: Single Store-Single Item with Holiday and Day of Week Considerations




```
GRU Test Loss : 0.00858777614593506
GRU Test MAE  : 0.05062335729598999
GRU Test RMSE : 0.0926702618598938
```

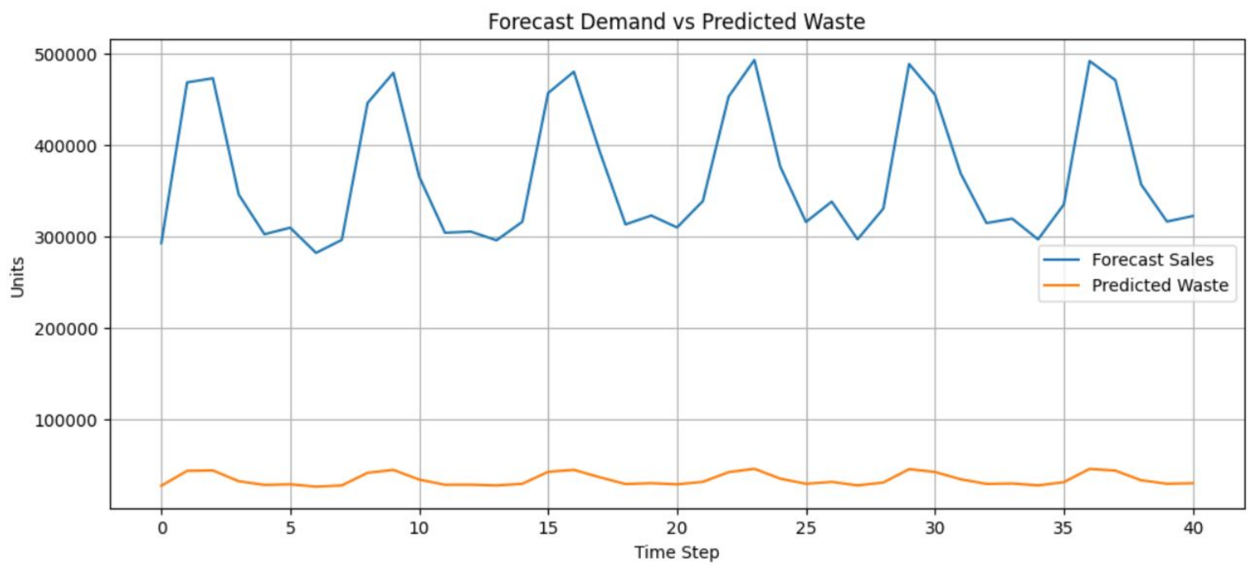
Recommended Inventory Order

```python
# ==========================================
# ORDER RECOMMENDATION MODULE
# RMSE-BASED SAFETY STOCK
# ==========================================

inventory_ratio = 0.75

inventory_available = actual_sales * inventory_ratio

# Safety stock based on forecast uncertainty
safety_stock = gru_results[2] * preds_actual

recommended_order = np.maximum(
    0,
    preds_actual + safety_stock - inventory_available
)

print("Average Safety Stock:",
      np.mean(safety_stock))

print("Average Recommended Order:",
      np.mean(recommended_order))
```

```
Average Safety Stock: 33974.582
Average Recommended Order: 128807.85815243902
```

Results (Part 3)

Forecasted Demand vs Predicted Waste



- Predicted food waste increases during periods of high demand.
- Higher demand results in larger inventory recommendations.
- Safety stock is calculated using the GRU model's RMSE.
- Additional inventory helps protect against forecast uncertainty.
- Reduces the risk of stock shortages during peak sales periods.
- Demonstrates the tradeoff between inventory availability and food waste.

Conclusion

- Key findings
  - MLP provided a useful baseline but reached its performance capacity
  - GRU significantly improved forecasting performance by learning sequential relationship data
  - Adding holiday and day of week features improved forecast accuracy
- Takeaways
  - Forecast uncertainty can be incorporated into inventory decisions using and adaptive recommendation framework to be validated before scaling
  - Starting with a single store and single item allowed forecasting and inventory recommendation framework to be validated before scaling

Future Work + Q&A

- Improvements
  - Add more conditions: weather and promotions
- Expand GRU
  - Multiple Stores
  - Multiple items
- Thank You